

Contents

1. Introduction.....	4
2. Exploratory Data Analysis (EDA).....	4
2.1 Display features.....	4
2.2 Display target.....	5
2.3 Target class inspection.....	6
2.4 Missing values in features.....	6
2.5 Missing values in target.....	7
2.6 Feature summary.....	8
2.7 Split numerical and categorical.....	9
2.8 Display categorical one-hot block.....	9
2.9 Numerical histograms.....	10
2.10 Correlation ranking plots.....	11
3. Data Preprocessing.....	14
3.1 Isolation Forest outlier removal.....	14
3.2 Display cleaned splits.....	14
3.3 Select top correlated features.....	15
3.4 Scale numerical, combine and split train/test.....	16
4. Baseline MLP.....	17
4.1 Define baseline MLP architecture.....	17
4.2 Train baseline with early stopping.....	18
4.3 Plot baseline loss curve.....	19
4.4 Baseline evaluation.....	22
5. Hyperparameter Exploration.....	23
5.1 Create tuning subset (100k samples).....	23
5.2 Split tuning train and validation.....	23
5.3 Reusable model builder.....	24
5.4 SGD tuning.....	25
5.4.1 Tune SGD learning rate.....	25
5.4.2 Tune SGD batch size.....	26
5.4.3 Tune SGD epochs.....	27

5.4.4 Train final SGD on full data	27
5.5 Adam tuning	28
5.5.1 Tune Adam learning rate	28
5.5.2 Tune Adam batch size	29
5.5.3 Tune Adam epochs	30
5.5.4 Train final Adam on full data	31
6. Comparison and Best Individual Model	32
6.1 Compare SGD vs Adam validation peaks	32
6.2 Compare Training Time for SGD and Adam	33
6.3 Training and loss curve for SGD	36
6.4 Training and loss curve for Adam	38
6.5 Save and reload models	40
7. Voting Ensemble (Part 3 Extension)	40
7.1 Generate predictions and probabilities	40
7.2 Hard voting ensemble	41
7.3 Soft voting ensemble	41
7.4 Accuracy comparison across models and ensembles	42
7.8 Identify best individual MLP and select its predictions	43
7.9 Confusion matrix heatmap and values	44
8. Discussion	46
8.1 Impact of Hyperparameter Changes	46
8.2 Strengths and Limitations	47
9. Conclusion	48

1. Introduction

The solution begins with the roadmap of: EDA → preprocessing → baseline MLP → hyperparameter tuning → ensemble voting.

To enable direct and reproducible access to the dataset, the required dependency is installed at the start of the notebook:

```
pip install ucimlrepo
```

The ucimlrepo package provides an interface to datasets hosted in the UCI Machine Learning Repository, which allows the notebook to retrieve data programmatically.

The dataset is subsequently loaded and verified using the following code:

```
from ucimlrepo import fetch_ucirepo
covertypes = fetch_ucirepo(id=31)
X = covertypes.data.features
y = covertypes.data.targets
print(covertypes.metadata)
print(covertypes.variables)
```

This step downloads the **Forest CoverType** dataset directly from UCI via `fetch_ucirepo(id=31)`, ensuring the correct source and version are used. The features and target are then separated into `X` and `y`, where `X` contains the 54 cartographic and environmental predictors and `y` contains the categorical label `Cover_Type`. By printing both `covertypes.metadata` and `covertypes.variables`, the notebook confirms key dataset properties such as scale, feature definitions, and class structure.

2. Exploratory Data Analysis (EDA)

2.1 Display features

```
X
```

Displaying X prints the complete feature matrix of the Forest CoverType dataset. The output confirms the dataset scale at 581,012 rows by 54 columns, consistent with the published specification

	Elevation	Aspect	Slope	Horizontal_Distance_To_Hydrology	Vertical_Distance_To_Hydrology	Horizontal_Distance_To_Roadways	Hillshade_9am	Hillshade_Noon	Hillshade_3pm	Horizo
0	2596	51	3	258	0	510	221	232	148	
1	2590	56	2	212	-6	390	220	235	151	
2	2804	139	9	268	65	3180	234	238	135	
3	2785	155	18	242	118	3090	238	238	122	
4	2595	45	2	153	-1	391	220	234	150	
...	...	...	...	...	...	...	...	...	...	
581007	2396	153	20	85	17	108	240	237	118	
581008	2391	152	19	67	12	95	240	237	119	
581009	2386	159	17	60	7	90	236	241	130	
581010	2384	170	15	60	5	90	230	245	143	
581011	2383	165	13	60	4	67	231	244	141	

Figure 1. The Feature Matrix of the Forest Cover Type

2.2 Display target

y

Cover_Type	
0	5
1	5
2	2
3	2
4	5
...	...
581007	3
581008	3
581009	3
581010	3
581011	3
581012 rows x 1 columns	

Figure 2 Target data

Printing `y` shows the target data as a single column labelled `Cover_Type`. The observed labels range from 1 through 7, confirming that the task is a seven-class classification problem.

2.3 Target class inspection

```
y_series = y['Cover_Type']  
print(y_series.unique())  
print(y_series.value_counts())
```

```
[5 2 1 7 3 6 4]  
Cover_Type  
2    283301  
1    211840  
3     35754  
7     20510  
6     17367  
5       9493  
4       2747  
Name: count, dtype: int64
```

Figure 3. Label Space and its Distribution are Validated

The label space and its distribution are validated. The `unique()` output confirms that all seven cover types are present, which is essential for ensuring a complete multi-class learning setup. The `value_counts()` results reveal substantial class imbalance, with Classes 1 and 2 contributing the largest share of samples, and Classes 4–7 appearing far less frequently.

Identifying this imbalance during EDA is important as it can influence model bias and evaluation.

2.4 Missing values in features

```
X.isnull().sum()
```

	0
Elevation	0
Aspect	0
Slope	0
Horizontal_Distance_To_Hydrology	0
Vertical_Distance_To_Hydrology	0
Horizontal_Distance_To_Roadways	0
Hillshade_9am	0
Hillshade_Noon	0
Hillshade_3pm	0
Horizontal_Distance_To_Fire_Points	0
Wilderness_Area1	0
Soil_Type1	0
Soil_Type2	0
Soil_Type3	0
Soil_Type4	0
Soil_Type5	0
Soil_Type6	0
Soil_Type7	0
Soil_Type8	0

Figure 4. Missing-value Summary

The missing-value summary returns zeros for every feature column, prove that the dataset contains no null entries among predictors. This confirms that feature imputation is unnecessary and allows preprocessing to focus on scaling and selection rather than data repair.

2.5 Missing values in target

```
y.isnull().sum()
```

```

0
Cover_Type 0
dtype: int64

```

Figure 5. Check the Target Missing Value

The target missing-value check also returns zero, confirming that every observation has a valid Cover_Type label. This ensures the dataset can be used in full without removing rows for incomplete supervision.

2.6 Feature summary

```
X.info()
```

```

#   Column      Non-Null Count  Dtype
---  -
0   Elevation    581012 non-null    int64
1   Aspect       581012 non-null    int64
2   Slope        581012 non-null    int64
3   Horizontal_Distance_To_Hydrology  581012 non-null    int64
4   Vertical_Distance_To_Hydrology    581012 non-null    int64
5   Horizontal_Distance_To_Roadways    581012 non-null    int64
6   Hillshade_9am  581012 non-null    int64
7   Hillshade_Noon 581012 non-null    int64
8   Hillshade_3pm  581012 non-null    int64
9   Horizontal_Distance_To_Fire_Points 581012 non-null    int64
10  Wilderness_Area1 581012 non-null    int64
11  Soil_Type1       581012 non-null    int64
12  Soil_Type2       581012 non-null    int64
13  Soil_Type3       581012 non-null    int64
14  Soil_Type4       581012 non-null    int64
15  Soil_Type5       581012 non-null    int64
16  Soil_Type6       581012 non-null    int64
17  Soil_Type7       581012 non-null    int64
18  Soil_Type8       581012 non-null    int64
19  Soil_Type9       581012 non-null    int64
20  Soil_Type10      581012 non-null    int64
21  Soil_Type11      581012 non-null    int64
22  Soil_Type12      581012 non-null    int64
23  Soil_Type13      581012 non-null    int64
24  Soil_Type14      581012 non-null    int64
25  Soil_Type15      581012 non-null    int64
26  Soil_Type16      581012 non-null    int64
27  Soil_Type17      581012 non-null    int64
28  Soil_Type18      581012 non-null    int64
29  Soil_Type19      581012 non-null    int64
30  Soil_Type20      581012 non-null    int64
31  Soil_Type21      581012 non-null    int64
32  Soil_Type22      581012 non-null    int64
33  Soil_Type23      581012 non-null    int64
34  Soil_Type24      581012 non-null    int64
35  Soil_Type25      581012 non-null    int64
36  Soil_Type26      581012 non-null    int64
37  Soil_Type27      581012 non-null    int64
38  Soil_Type28      581012 non-null    int64

```

Figure 6. Info of the data

The info() output provides a concise structural audit of the predictors. It confirms the presence of ten numerical variables and forty-four binary columns corresponding to one-hot categorical encodings.

2.7 Split numerical and categorical

```
X_num = X.iloc[:, :10]
X_cat = X.iloc[:, 10:]
print("X Numerical")
X_num
```

X Numerical										
	Elevation	Aspect	Slope	Horizontal_Distance_To_Hydrology	Vertical_Distance_To_Hydrology	Horizontal_Distance_To_Roadways	Hillshade_9am	Hillshade Noon	Hillshade_3pm	Horizontal_Distance_To_Fire_Points
0	2596	51	3	258	0	510	221	232	148	6279
1	2590	56	2	212	-6	390	220	235	151	6225
2	2804	139	9	268	65	3180	234	238	135	6121
3	2785	155	18	242	118	3090	238	238	122	6211
4	2595	45	2	153	-1	391	220	234	150	6172
...	...	...	...	...	...	...	...	...	...	...
581007	2396	153	20	85	17	108	240	237	118	837
581008	2391	152	19	67	12	95	240	237	119	845
581009	2386	159	17	60	7	90	236	241	130	854
581010	2384	170	15	60	5	90	230	245	143	864
581011	2383	165	13	60	4	67	231	244	141	875

581012 rows x 10 columns

Figure 7. Feature Matrix for Numerical Block and Categorical Block

he feature matrix is partitioned by column index into a numerical block (X_num) and a categorical block (X_cat). Displaying X_num confirms that it contains exactly the first ten continuous predictors. The split enables targeted preprocessing, ensure the numerical scaling is applied only for the numerical block.

2.8 Display categorical one-hot block

```
print("X Categorical")
X_cat
```

X Categorical																		
	Wilderness_Area1	Soil_Type1	Soil_Type2	Soil_Type3	Soil_Type4	Soil_Type5	Soil_Type6	Soil_Type7	Soil_Type8	Soil_Type9	...	Soil_Type34	Soil_Type35	Soil_Type36	Soil_Type37	Soil_Type38	Soil_Type39	
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
1	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
3	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
4	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
581007	0	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
581008	0	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
581009	0	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
581010	0	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
581011	0	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	
581012 rows x 44 columns																		

Figure 8. Showing X Category are all binary one-hot

Printing X_cat verifies that the remaining columns are binary one-hot features. These correspond to Wilderness Area and Soil Type categories, already encoded as 0/1 indicators. This confirms that no further categorical encoding is required, and that later preprocessing

can preserve these variables unchanged except for optional feature selection.

2.9 Numerical histograms

```
X_num.hist(figsize=(15,15))  
plt.show()
```

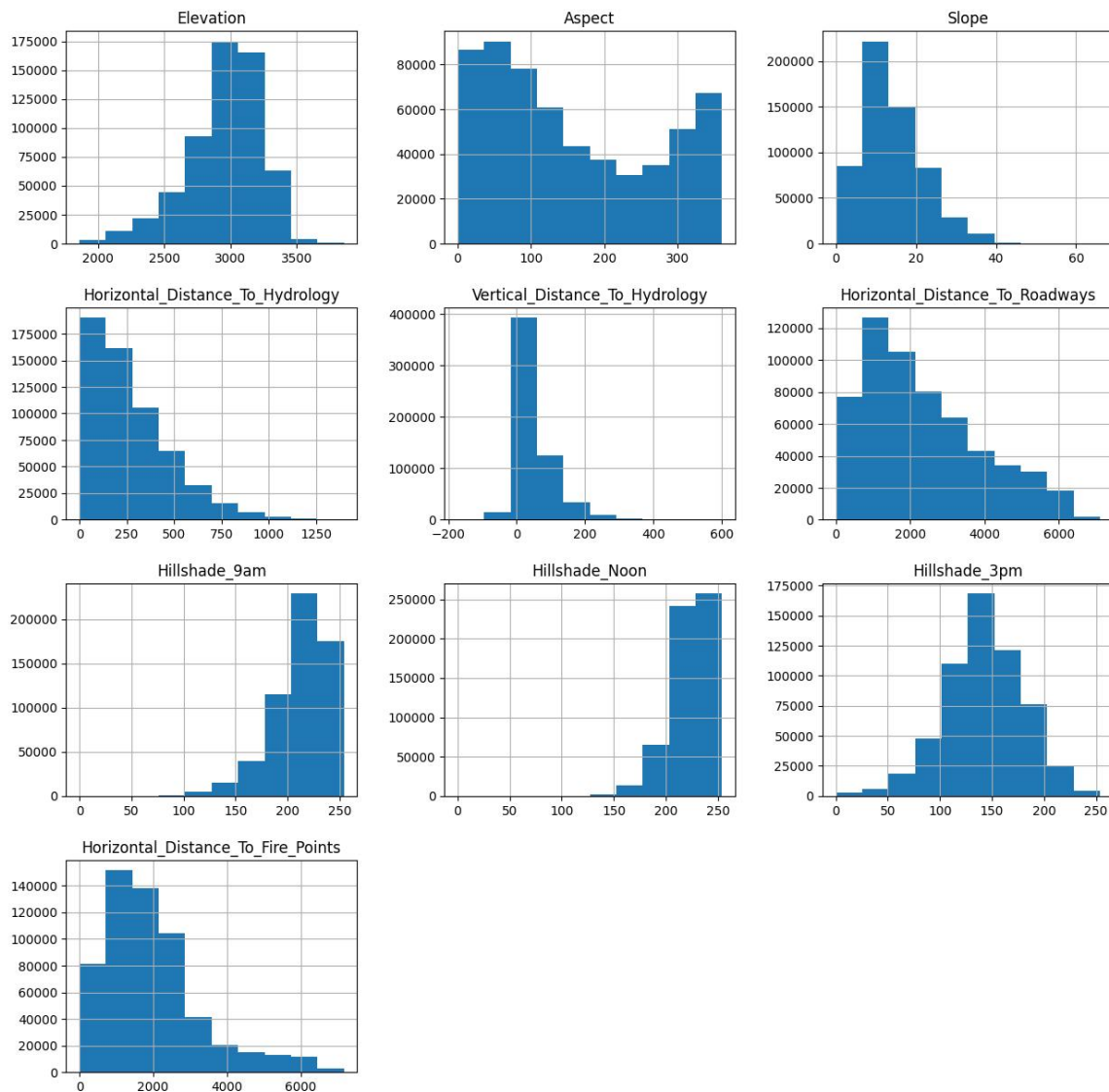


Figure 9. Numerical Histogram

The histogram grid visualises the distribution of each numerical feature. The plots show that many distance-related variables are strongly right-skewed, with long tails indicating a small number of extreme values.

Feature ranges also vary substantially, as elevation spanning thousands of units while slope and hillshade measures occupy much smaller scales.

2.10 Correlation ranking plots

```
# Ensure 1D target
y_1d = y.squeeze()

# Combine features
X_all = pd.concat([X_num, X_cat], axis=1)

# Compute absolute correlation with target
corr_with_target = X_all.join(y_1d.rename('Target')).corr()['Target'].abs()
corr_with_target = corr_with_target.drop('Target')

# Separate numerical vs categorical
num_corr = corr_with_target[X_num.columns].sort_values(ascending=False)
cat_corr = corr_with_target[X_cat.columns].sort_values(ascending=False)

# Plot
fig, axes = plt.subplots(2, 1, figsize=(15,15))
```

```
# Numerical features
```

```
sns.barplot(x=num_corr.index, y=num_corr.values, ax=axes[0],  
color='skyblue')
```

```
axes[0].set_ylabel("Abs Corr with Target")
```

```
axes[0].set_xlabel("Numerical Features")
```

```
axes[0].set_title("Numerical Feature Correlation Ranking")
```

```
axes[0].tick_params(axis='x', rotation=90)
```

```
# Categorical features
```

```
sns.barplot(x=cat_corr.index, y=cat_corr.values, ax=axes[1],  
color='lightgreen')
```

```
axes[1].set_ylabel("Abs Corr with Target")
```

```
axes[1].set_xlabel("Categorical Features")
```

```
axes[1].set_title("Categorical One-hot Feature Correlation Ranking")
```

```
axes[1].tick_params(axis='x', rotation=90)
```

```
plt.tight_layout()
```

```
plt.show()
```

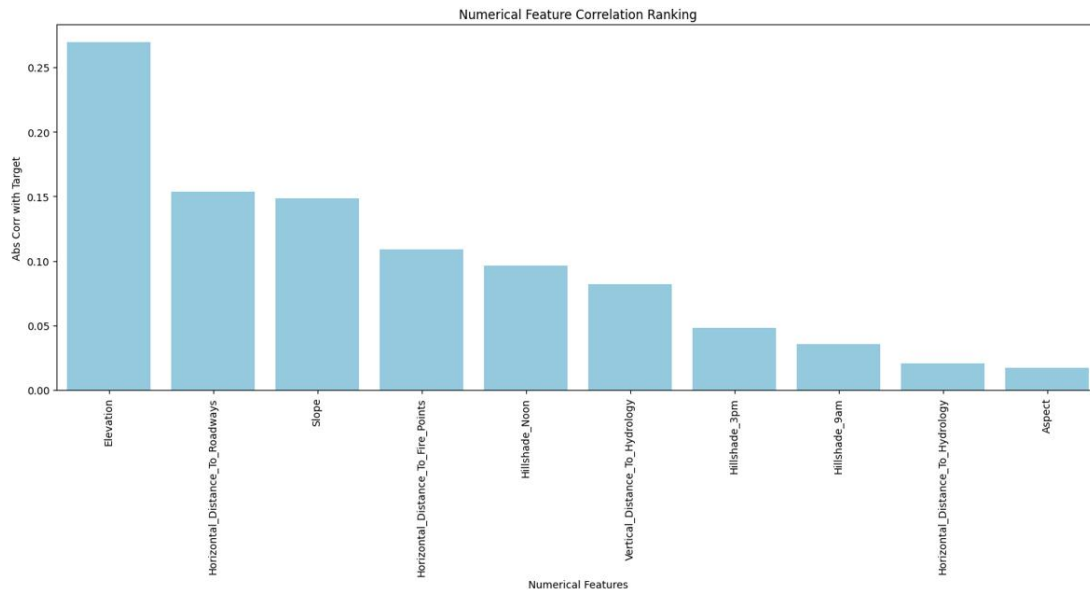


Figure 10. Numerical Feature Correlation Ranking

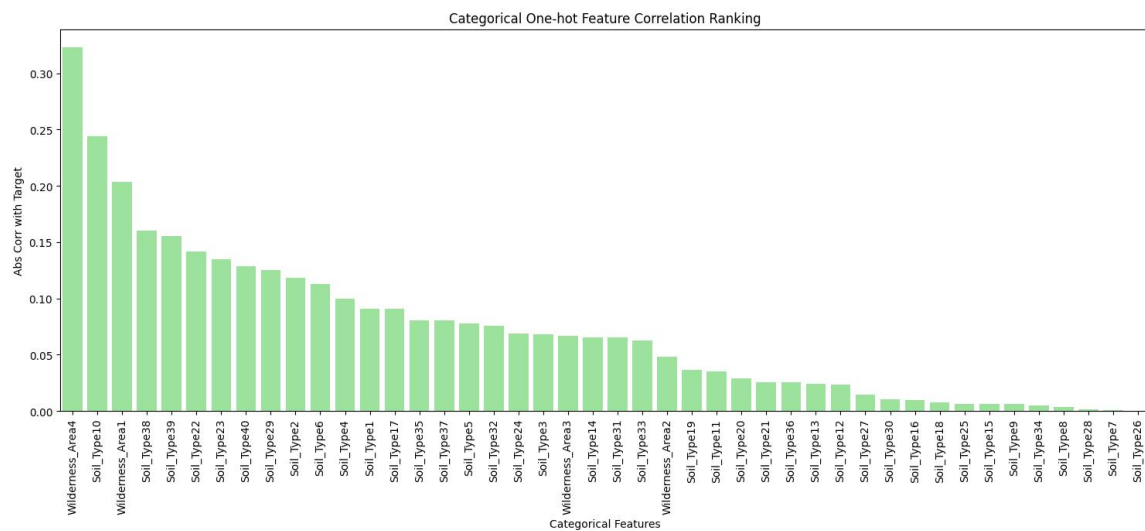


Figure 11. Categorical One-hot Feature Correlation Ranking

Linear relationships between each feature and the target is shown, then ranks them separately for numerical and categorical groups. The numerical ranking highlights elevation as the most correlated continuous predictor, while the categorical ranking identifies Wilderness_Area4 as the strongest one-hot contributor.

The broader pattern shows that only a subset of predictors exhibits meaningful correlation, providing empirical motivation for correlation-

guided feature selection or dimensionality reduction to reduce redundancy and improve downstream model efficiency.

3. Data Preprocessing

3.1 Isolation Forest outlier removal

```
from sklearn.ensemble import IsolationForest
import pandas as pd

X_combined = pd.concat([X_num, X_cat], axis=1)

iso = IsolationForest(
    n_estimators=300,
    contamination=0.01,
    random_state=42
)
iso.fit(X_combined)

labels = iso.predict(X_combined) # -1 = outlier, 1 = normal
mask = labels == 1

X_num_no_outliers = X_num[mask].copy()
X_cat_no_outliers = X_cat[mask].copy()
y_no_outliers = y[mask].copy()
```

Isolation Forest is used for outlier detection and removal, an unsupervised method suited for high-dimensional data. By first concatenating numerical and categorical blocks into `X_combined`, the algorithm evaluates anomalies across the full feature space. The model is configured with 300 estimators for stability, and a contamination rate of 0.01 to remove approximately 1% of observations identified as extreme.

After fitting, predictions label each sample as normal (1) or outlier (-1), and a boolean mask filters the dataset accordingly.

3.2 Display cleaned splits

```
X_num_no_outliers
```

```
X_cat_no_outliers
y_no_outliers
```

The filtered numerical features, categorical one-hot features, and target labels after applying the Isolation Forest mask are displayed. The outputs confirm that the removal step executed correctly, with datasets retaining the same feature structure as before but containing fewer rows.

3.3 Select top correlated features

```
X_no_outliers = pd.concat([X_num_no_outliers, X_cat_no_outliers],
axis=1)
y_no_outliers = y_no_outliers.squeeze()

top_N = 20
top_features = corr_with_target.index[:top_N]

num_sel = [f for f in top_features if f in X_num_no_outliers.columns]
cat_sel = [f for f in top_features if f in X_cat_no_outliers.columns]

print("Top numerical features selected:", num_sel)
print("Top categorical features selected:", cat_sel)
```

Output:

```
Top numerical features selected: ['Elevation', 'Aspect', 'Slope',
'Horizontal_Distance_To_Hydrology',
'Vertical_Distance_To_Hydrology',
'Horizontal_Distance_To_Roadways', 'Hillshade_9am',
'Hillshade_Noon', 'Hillshade_3pm',
'Horizontal_Distance_To_Fire_Points']

Top categorical features selected: ['Wilderness_Area1', 'Soil_Type1',
'Soil_Type2', 'Soil_Type3', 'Soil_Type4', 'Soil_Type5', 'Soil_Type6',
'Soil_Type7', 'Soil_Type8', 'Soil_Type9']
```

The top 20 predictors are selected based on the correlation rankings. Numerical and categorical features are recombined into

X_no_outliers to allow unified indexing, while the target is squeezed into a one-dimensional structure suitable for downstream modelling.

By using `corr_with_target`, the code extracts the highest-ranked features, then separates them back into numerical selections (`num_sel`) and categorical selections (`cat_sel`) depending on their original blocks.

3.4 Scale numerical, combine and split train/test

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import numpy as np

scaler = StandardScaler()
X_num_scaled = scaler.fit_transform(X_num_no_outliers[num_sel])

X_cat_sel = X_cat_no_outliers[cat_sel].values
X_sc = np.hstack([X_num_scaled, X_cat_sel])

X_train, X_test, y_train, y_test = train_test_split(
    X_sc, y_no_outliers, test_size=0.2, random_state=42
)
```

```
Preprocessing complete. Shapes:
X_train: (460160, 20)
X_test : (115041, 20)
y_train: (460160,)
y_test : (115041,)
```

Figure 12. Pre-processing Complete

The selected numerical variables are standardised, and the final modelling matrix is constructed. `StandardScaler` is fitted only on the numerical subset because these features exhibit wide range differences and skewness, as observed in the histograms; scaling prevents high-magnitude variables from dominating gradient updates in the MLP. The categorical one-hot indicators are extracted without scaling since they are already binary 0/1 values. Both blocks are then horizontally stacked into `X_sc`, producing a unified feature array with consistent ordering. Finally, `train_test_split` divides the cleaned and

scaled dataset into an 80/20 split with a fixed random state for reproducibility.

4. Baseline MLP

4.1 Define baseline MLP architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model_base = Sequential([
    Dense(256, activation='relu', input_dim=X_train.shape[1]),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(8, activation='softmax')
])
```

```
model_base.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model_base.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	5,376
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 8)	520

Total params: 47,048 (183.78 KB)
Trainable params: 47,048 (183.78 KB)
Non-trainable params: 0 (0.00 B)

Figure 13. Baseline MLP Architecture

This baseline multilayer perceptron using a straightforward feed-forward architecture. The network contains three hidden layers with

decreasing widths ($256 \rightarrow 128 \rightarrow 64$), each using ReLU to introduce nonlinearity and allow the model to learn complex decision boundaries.

The output layer employs an eight-unit softmax, matching the model's multi-class prediction goal. The Adam optimizer is used during compilation to ensure stable gradient updates, and sparse categorical cross-entropy is chosen because the target labels are integer-encoded rather than one-hot encoded.

4.2 Train baseline with early stopping

```
from tensorflow.keras.callbacks import EarlyStopping

early_stp = EarlyStopping(monitor='val_loss', patience=2,
                           restore_best_weights=True)

history_base = model_base.fit(
    X_train, y_train,
    epochs=23, batch_size=32,
    callbacks=[early_stp],
    validation_split=0.2
)
```

```

Epoch 1/100
11504/11504 - 33s - 3ms/step - accuracy: 0.7620 - loss: 0.5509 - val_accuracy: 0.8008 - val_loss: 0.4643
Epoch 2/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8158 - loss: 0.4319 - val_accuracy: 0.8317 - val_loss: 0.4012
Epoch 3/100
11504/11504 - 31s - 3ms/step - accuracy: 0.8404 - loss: 0.3784 - val_accuracy: 0.8413 - val_loss: 0.3733
Epoch 4/100
11504/11504 - 31s - 3ms/step - accuracy: 0.8569 - loss: 0.3435 - val_accuracy: 0.8636 - val_loss: 0.3295
Epoch 5/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8667 - loss: 0.3203 - val_accuracy: 0.8700 - val_loss: 0.3123
Epoch 6/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8745 - loss: 0.3022 - val_accuracy: 0.8697 - val_loss: 0.3159
Epoch 7/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8819 - loss: 0.2865 - val_accuracy: 0.8786 - val_loss: 0.2959
Epoch 8/100
11504/11504 - 29s - 3ms/step - accuracy: 0.8863 - loss: 0.2753 - val_accuracy: 0.8799 - val_loss: 0.2946
Epoch 9/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8905 - loss: 0.2649 - val_accuracy: 0.8855 - val_loss: 0.2793
Epoch 10/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8943 - loss: 0.2571 - val_accuracy: 0.8900 - val_loss: 0.2671
Epoch 11/100
11504/11504 - 30s - 3ms/step - accuracy: 0.8976 - loss: 0.2496 - val_accuracy: 0.8920 - val_loss: 0.2649
Epoch 12/100
11504/11504 - 31s - 3ms/step - accuracy: 0.9005 - loss: 0.2427 - val_accuracy: 0.8943 - val_loss: 0.2630
Epoch 13/100
11504/11504 - 30s - 3ms/step - accuracy: 0.9033 - loss: 0.2373 - val_accuracy: 0.8981 - val_loss: 0.2510
Epoch 14/100
11504/11504 - 30s - 3ms/step - accuracy: 0.9051 - loss: 0.2315 - val_accuracy: 0.8989 - val_loss: 0.2495
Epoch 15/100
11504/11504 - 30s - 3ms/step - accuracy: 0.9071 - loss: 0.2271 - val_accuracy: 0.9054 - val_loss: 0.2326
Epoch 16/100
11504/11504 - 30s - 3ms/step - accuracy: 0.9094 - loss: 0.2222 - val_accuracy: 0.8990 - val_loss: 0.2500
Epoch 17/100
11504/11504 - 30s - 3ms/step - accuracy: 0.9110 - loss: 0.2180 - val_accuracy: 0.9074 - val_loss: 0.2279
Epoch 18/100
11504/11504 - 30s - 3ms/step - accuracy: 0.9127 - loss: 0.2149 - val_accuracy: 0.9090 - val_loss: 0.2289
Epoch 19/100
11504/11504 - 29s - 3ms/step - accuracy: 0.9137 - loss: 0.2108 - val_accuracy: 0.9084 - val_loss: 0.2328

```

Figure 14. MLP Training Using Early Stopping

Training is carried out with early stopping to prevent overfitting and unnecessary computation power. By monitoring validation loss and using a patience of two epochs, the callback halts training once performance stops improving, while `restore_best_weights=True` ensures the final model corresponds to the best validation point rather than the last epoch.

The fit procedure uses a batch size of 32 and allows up to 23 epochs, with 20% of the training data held out for validation. Saving the returned `history_base` object records learning curves for later analysis, meeting the requirement to document training behaviour.

4.3 Plot baseline loss curve

```

import matplotlib.pyplot as plt

# Plot training & validation accuracy

```

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('accuracy_plot.png') # Save as PNG
plt.show()
```

```
# Plot training & validation loss
```

```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.savefig('loss_plot.png') # Save as PNG
plt.show()
```

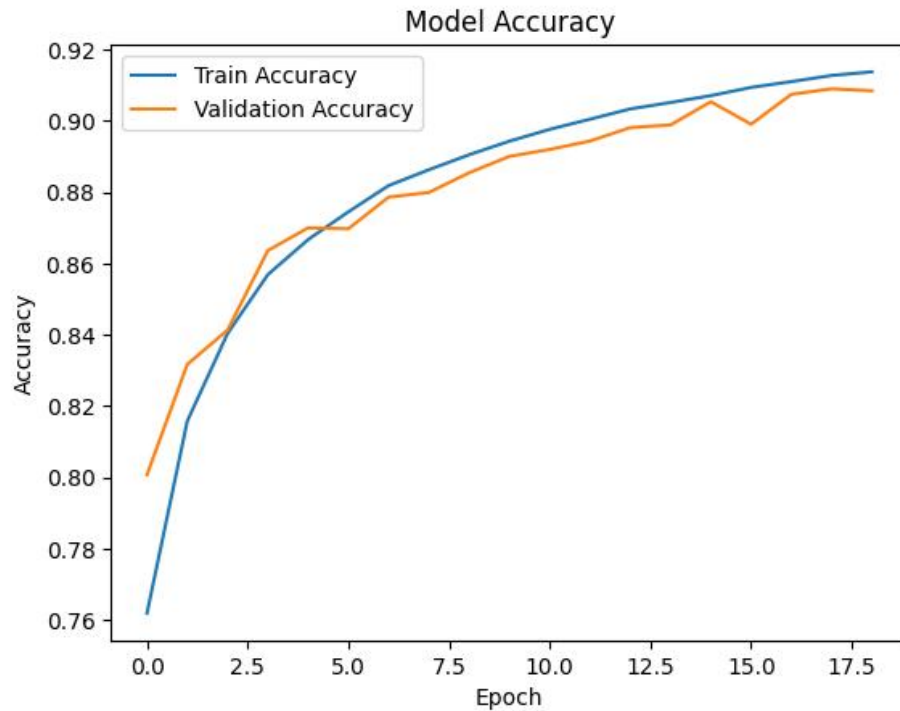


Figure 15. Modal Accuracy of Baseline MLP

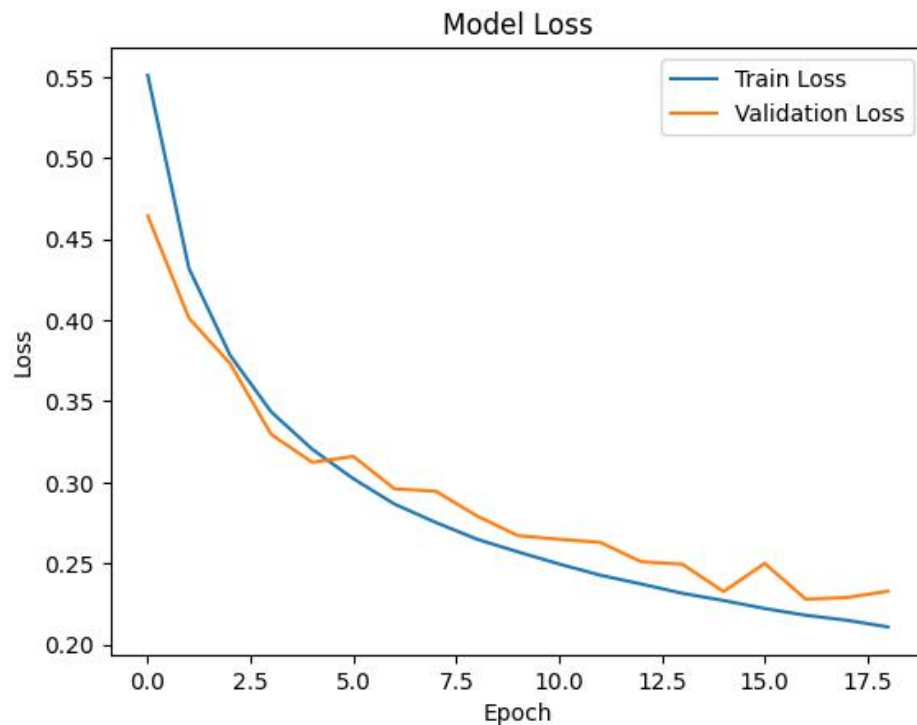


Figure 16. Modal Loss for Baseline MLP

The resulting curve shows rapid early convergence, indicating that the scaled and selected features provide a learnable signal for the MLP. The small and stable gap between training and validation loss

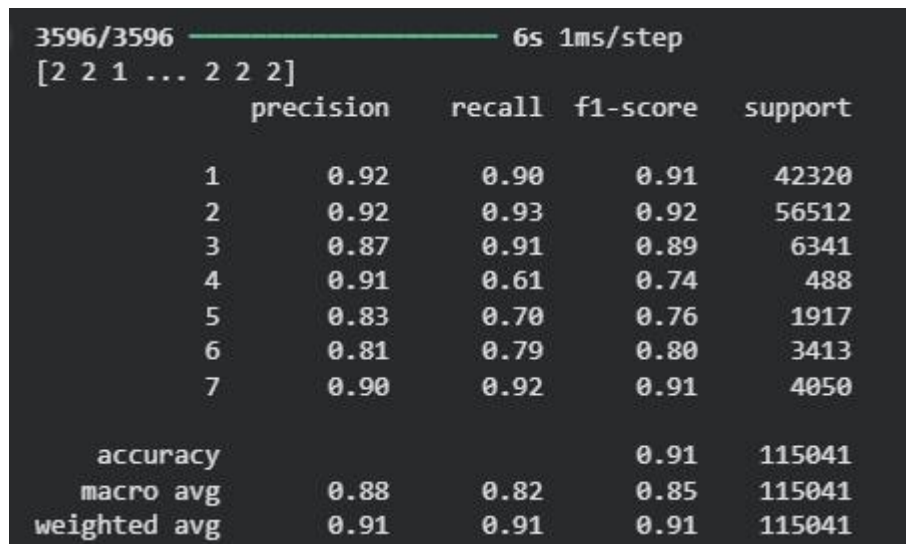
suggests reasonable generalisation, with no strong evidence of divergence or severe overfitting at the baseline stage.

4.4 Baseline evaluation

```
import numpy as np

y_pred = model.predict(X_test)
y_pred = np.argmax(y_pred, axis=1)
print(y_pred)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```



3596/3596 ————— 6s 1ms/step				
[2 2 1 ... 2 2 2]				
	precision	recall	f1-score	support
1	0.92	0.90	0.91	42320
2	0.92	0.93	0.92	56512
3	0.87	0.91	0.89	6341
4	0.91	0.61	0.74	488
5	0.83	0.70	0.76	1917
6	0.81	0.79	0.80	3413
7	0.90	0.92	0.91	4050
accuracy			0.91	115041
macro avg	0.88	0.82	0.85	115041
weighted avg	0.91	0.91	0.91	115041

Figure 17. Baseline Evaluation

The baseline model is evaluated on the held-out test set using a full classification report. Predictions are generated as class probabilities, then converted to discrete labels via `argmax`.

Reporting precision, recall, and F1-score per class provides a detailed view of baseline strengths and weaknesses, which is particularly important given the earlier-identified class imbalance.

5. Hyperparameter Exploration

5.1 Create tuning subset (100k samples)

```
TUNE_SIZE = 100000
X_tune = X_no_outliers.sample(TUNE_SIZE, random_state=42)
y_tune = y_no_outliers.loc[X_tune.index]
```

A reduced dataset is constructed for hyperparameter search by sampling 100,000 observations from the cleaned feature matrix.

5.2 Split tuning train and validation

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

# Separate numerical & categorical features for tuning subset
X_tune_num = X_tune[num_sel]
X_tune_cat = X_tune[cat_sel]

# Scale only numerical features
scaler_tune = StandardScaler()
X_tune_num_sc = scaler_tune.fit_transform(X_tune_num)

# Combine scaled numerical + categorical
X_tune_sc = np.hstack([X_tune_num_sc, X_tune_cat.values])
X_tune_train, X_tune_val, y_tune_train, y_tune_val = train_test_split(
    X_tune_sc, y_tune, test_size=0.2, random_state=42)
```

```
)
print(X_tune_train.shape, X_tune_val.shape)
```

```
(80000, 20) (20000, 20)
```

Figure 18. Train Validation Split

The tuning subset is divided into training and validation partitions in an 80%/20% split.

5.3 Reusable model builder

```
import tensorflow as tf
from tensorflow.keras.optimizers import SGD, Adam

def build_model(optimizer, lr):
    if optimizer == "sgd":
        opt = SGD(learning_rate=lr)
    elif optimizer == "adam":
        opt = Adam(learning_rate=lr)

    model = Sequential([
        Dense(256, activation='relu', input_dim=X_tune_train.shape[1]),
        Dense(128, activation='relu'),
        Dense(64, activation='relu'),
        Dense(8, activation='softmax')
    ])

    model.compile(optimizer=opt,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model
```

A reusable function that builds and compiles an MLP is defined while allowing key hyperparameters to vary. By keeping the network architecture fixed and changing only the optimiser type and learning rate, the notebook ensures fair, apples-to-apples comparison across trials.

The function selects either SGD or Adam based on input, applies the specified learning rate, and returns a compiled model ready for training on the tuning split. This design reduces code repetition, improves experimental clarity, and directly supports the goal of isolating the impact of hyperparameters on model performance.

5.4 SGD tuning

5.4.1 Tune SGD learning rate

```
sgd_lrs = [0.1, 0.01, 0.001]
sgd_lr_results = {}

for lr in sgd_lrs:
    model = build_model(optimizer="sgd", lr=lr)
    history = model.fit(
        X_tune_train, y_tune_train,
        validation_data=(X_tune_val, y_tune_val),
        epochs=20, batch_size=32, verbose=0
    )
    best_val_acc = max(history.history["val_accuracy"])
    sgd_lr_results[lr] = best_val_acc
    print(f"SGD lr={lr} best val_accuracy={best_val_acc:.4f}")

best_sgd_lr = max(sgd_lr_results, key=sgd_lr_results.get)
print("Best SGD learning rate:", best_sgd_lr)
```

```
SGD LR=0.1 → Best Val Acc = 0.8481
SGD LR=0.01 → Best Val Acc = 0.7939
SGD LR=0.001 → Best Val Acc = 0.7270
```

Figure 19. Learning Rate of SGD

```
Final SGD LR Results:
{0.1: 0.8481000065803528, 0.01: 0.7938500046730042, 0.001: 0.7269999980926514, 'best': {'lr': 0.1, 'val_accuracy': 0.8481000065803528}}
```

Figure 20. Final SDG LR Result

Several learning rates for SGD are evaluated by training identical models on the tuning split and tracking the best validation accuracy achieved in each run. Testing rates of 0.1, 0.01, and 0.001 provides coverage from aggressive to conservative step sizes.

Selecting the learning rate based on peak validation performance identifies the SGD configuration that most effectively balances convergence speed and stability for this dataset. As a result, learning rate with 0.1 is the best validation accuracy, which is 0.8481.

5.4.2 Tune SGD batch size

```
sgd_batches = [32, 64, 128]
sgd_batch_results = {}

for bs in sgd_batches:
    model = build_model(optimizer="sgd", lr=best_sgd_lr)
    history = model.fit(
        X_tune_train, y_tune_train,
        validation_data=(X_tune_val, y_tune_val),
        epochs=20, batch_size=bs, verbose=0
    )
    best_val_acc = max(history.history["val_accuracy"])
    sgd_batch_results[bs] = best_val_acc
    print(f"SGD batch_size={bs} best_val_accuracy={best_val_acc:.4f}")

best_sgd_bs = max(sgd_batch_results, key=sgd_batch_results.get)
print("Best SGD batch size:", best_sgd_bs)
```

```
SGD Batch=32 -> Best Val Acc = 0.8479
SGD Batch=64 -> Best Val Acc = 0.8343
SGD Batch=128 -> Best Val Acc = 0.8091

Final SGD Batch Size Results:
{32: 0.8478999733924866, 64: 0.8342999815940857, 128: 0.8090999722480774, 'best': {'batch_size': 32, 'val_accuracy': 0.8478999733924866}}
```

Figure 21. Batch Size of SDG

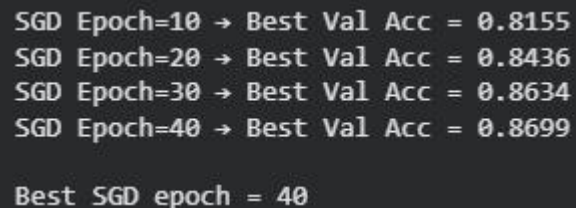
Batch sizes of 32, 64, and 128 are systematically compared using the previously selected optimal SGD learning rate. Because batch size affects the noisiness of gradient estimates and the frequency of parameter updates, it can materially influence generalisation and convergence. By choosing the batch size that yields the highest validation accuracy under otherwise identical conditions, the notebook identifies the minibatch regime most compatible with SGD on this problem. As a result, the best batch size for SDG Model is 32, with the accuracy of 0.8479.

5.4.3 Tune SGD epochs

```
sgd_epochs_list = [10, 20, 30, 40]
sgd_epoch_results = {}

for ep in sgd_epochs_list:
    model = build_model(optimizer="sgd", lr=best_sgd_lr)
    history = model.fit(
        X_tune_train, y_tune_train,
        validation_data=(X_tune_val, y_tune_val),
        epochs=ep, batch_size=best_sgd_bs, verbose=0
    )
    best_val_acc = max(history.history["val_accuracy"])
    sgd_epoch_results[ep] = best_val_acc
    print(f"SGD epochs={ep} best val_accuracy={best_val_acc:.4f}")

best_sgd_epochs = max(sgd_epoch_results,
key=sgd_epoch_results.get)
print("Best SGD epochs:", best_sgd_epochs)
```



```
SGD Epoch=10 → Best Val Acc = 0.8155
SGD Epoch=20 → Best Val Acc = 0.8436
SGD Epoch=30 → Best Val Acc = 0.8634
SGD Epoch=40 → Best Val Acc = 0.8699

Best SGD epoch = 40
```

Figure 22. Epoch for SDG

Here, the notebook tests multiple epoch budgets to determine how long SGD should train before validation gains plateau. Shorter runs risk underfitting, whereas longer runs may lead to diminishing returns or overfitting. Measuring the best validation accuracy across 10, 20, 30, and 40 epochs provides an empirical basis for selecting a training duration that best supports generalisation on unseen data. As a result, the best SDG epoch is 40, with the accuracy of 0.8699.

5.4.4 Train final SGD on full data

```
model_sgd_final = build_model(optimizer="sgd", lr=best_sgd_lr)
```

```

history_sgd_final = model_sgd_final.fit(
    X_tune_train, y_tune_train,
    validation_data=(X_tune_val, y_tune_val),
    epochs=best_sgd_epochs,
    batch_size=best_sgd_bs,
    verbose=1
)

```

```

Epoch 16/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.8993 - loss: 0.2464 - val_accuracy: 0.8981 - val_loss: 0.2528
Epoch 17/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9020 - loss: 0.2409 - val_accuracy: 0.9006 - val_loss: 0.2468
Epoch 18/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9037 - loss: 0.2366 - val_accuracy: 0.9002 - val_loss: 0.2464
Epoch 19/40
11504/11504 ————— 31s 3ms/step - accuracy: 0.9058 - loss: 0.2321 - val_accuracy: 0.9027 - val_loss: 0.2437
Epoch 20/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9078 - loss: 0.2275 - val_accuracy: 0.9028 - val_loss: 0.2425
Epoch 21/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9088 - loss: 0.2261 - val_accuracy: 0.9030 - val_loss: 0.2424
Epoch 22/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9098 - loss: 0.2238 - val_accuracy: 0.9003 - val_loss: 0.2507
Epoch 23/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9117 - loss: 0.2201 - val_accuracy: 0.9054 - val_loss: 0.2326
Epoch 24/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9116 - loss: 0.2177 - val_accuracy: 0.9081 - val_loss: 0.2274
Epoch 25/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9133 - loss: 0.2139 - val_accuracy: 0.9083 - val_loss: 0.2296
Epoch 26/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9141 - loss: 0.2130 - val_accuracy: 0.9053 - val_loss: 0.2368
Epoch 27/40
11504/11504 ————— 30s 3ms/step - accuracy: 0.9152 - loss: 0.2107 - val_accuracy: 0.9075 - val_loss: 0.2339

```

Figure 23. Training of SDG Model

The final SGD model is trained using the selected learning rate, batch size, and epoch count. The resulting history object preserves loss and accuracy trajectories for reporting and later visual comparison. The accompanying SGD loss curve provides a qualitative check on optimisation behaviour, confirming whether convergence is smooth and whether validation performance remains stable under the tuned settings. The training for SDG stops at epoch 27.

5.5 Adam tuning

5.5.1 Tune Adam learning rate

```

adam_lrs = [0.01, 0.001, 0.0001]
adam_lr_results = {}

for lr in adam_lrs:
    model = build_model(optimizer="adam", lr=lr)

```

```

history = model.fit(
    X_tune_train, y_tune_train,
    validation_data=(X_tune_val, y_tune_val),
    epochs=20, batch_size=32, verbose=0
)
best_val_acc = max(history.history["val_accuracy"])
adam_lr_results[lr] = best_val_acc
print(f"Adam lr={lr} best val_accuracy={best_val_acc:.4f}")

best_adam_lr = max(adam_lr_results, key=adam_lr_results.get)
print("Best Adam learning rate:", best_adam_lr)

```

```

Adam LR=0.01 → Best Val Acc = 0.8067
Adam LR=0.001 → Best Val Acc = 0.8630
Adam LR=0.0001 → Best Val Acc = 0.7947

```

Figure 24. Adam Learning Rate

```

Final Adam LR Results:
{0.01: 0.8066999912261963, 0.001: 0.8629999756813049, 0.0001: 0.794700026512146, 'best': {'lr': 0.001, 'val_accuracy': 0.8629999756813049}}

```

Figure 25. Best Adam Learning Rate

Three learning rates for the Adam optimiser are evaluated, ranging from relatively large (0.01) to small (0.0001). Each learning rate is tested by training a model on the tuning training split and recording the maximum validation accuracy achieved.

Selecting the rate that yields the highest validation accuracy identifies the most effective step size for stable and efficient optimisation on the CoverType tuning subset. As a result, the best learning rate is 0.01, with the accuracy of 0.8630.

5.5.2 Tune Adam batch size

```

adam_batches = [32, 64, 128]
adam_batch_results = {}

for bs in adam_batches:
    model = build_model(optimizer="adam", lr=best_adam_lr)
    history = model.fit(
        X_tune_train, y_tune_train,

```

```

        validation_data=(X_tune_val, y_tune_val),
        epochs=20, batch_size=bs, verbose=0
    )
    best_val_acc = max(history.history["val_accuracy"])
    adam_batch_results[bs] = best_val_acc
    print(f"Adam          batch_size={bs}          best
val_accuracy={best_val_acc:.4f}")

best_adam_bs          =          max(adam_batch_results,
key=adam_batch_results.get)
print("Best Adam batch size:", best_adam_bs)

```

```

Adam Batch=32 → Best Val Acc = 0.8587
Adam Batch=64 → Best Val Acc = 0.8527
Adam Batch=128 → Best Val Acc = 0.8409

```

Figure 26 Batch Size for Adam

```

Final Adam Batch Size Results:
{32: 0.8587499856948853, 64: 0.8526999950408936, 128: 0.8409000039100647, 'best': {'batch_size': 32, 'val_accuracy': 0.8587499856948853}}

```

Figure 27. Adam Batch Size Result

Batch sizes of 32, 64, and 128 are compared while keeping the Adam learning rate fixed at its tuned value. Batch size influences the trade-off between gradient noise and computational efficiency, which can affect both convergence dynamics and generalisation. The best batch size is 32, with the accuracy of 0.8587.

5.5.3 Tune Adam epochs

```

# Choose the best epoch count
# Use 100k dataset
adam_epochs = [10, 20, 30, 40]
adam_epoch_results = {}

# Get the best batch size from previous tuning
best_bs_adam = adam_bs_results['best']['batch_size']

for ep in adam_epochs:
    model = build_model("adam", best_lr_adam)
    history = model.fit(

```

```

X_tune_train, y_tune_train,
epochs=ep, batch_size=best_bs_adam, verbose=0,
validation_data=(X_tune_val, y_tune_val)
)
acc = max(history.history['val_accuracy'])
adam_epoch_results[ep] = acc
print(f"Adam Epoch={ep} → Best Val Acc = {acc:.4f}")

# Automatically determine the best epoch count
best_epoch_adam = max(adam_epoch_results,
key=adam_epoch_results.get)
best_epoch_acc = adam_epoch_results[best_epoch_adam]

# Add the best info into the results dictionary
adam_epoch_results['best'] = {'epochs': best_epoch_adam,
'val_accuracy': best_epoch_acc}

print("\nFinal Adam Epoch Results:")
print(adam_epoch_results)

```

```

Adam Epoch=10 → Best Val Acc = 0.8360
Adam Epoch=20 → Best Val Acc = 0.8558
Adam Epoch=30 → Best Val Acc = 0.8713
Adam Epoch=40 → Best Val Acc = 0.8841

```

Figure 28. Epoch for Adam

```

Final Adam Epoch Results:
{10: 0.8360499739646012, 20: 0.8557999730110168, 30: 0.8713499903678894, 40: 0.8841000199317932, 'best': {'epochs': 40, 'val_accuracy': 0.8841000199317932}}

```

Figure 29. Best epoch Result for Adam

Here, the notebook tests four epoch budgets to locate an empirically optimal training length under Adam. Short runs may underfit, while longer runs can yield diminishing returns or begin to overfit even with adaptive optimisation. The best epoch count is 40, which is 0.8841.

5.5.4 Train final Adam on full data

```

# Get best hyperparameters from tuning results
best_bs_adam = adam_bs_results['best']['batch_size']

```

```

best_epoch_adam = adam_epoch_results['best']['epochs']

# Build final Adam model
model_adam_final = build_model("adam", best_lr_adam)

# Train on full dataset with early stopping
history_adam = model_adam_final.fit(
    X_train_full, y_train_full,
    epochs=best_epoch_adam,
    batch_size=best_bs_adam,
    validation_split=0.2,
    callbacks=[es],
    verbose=1
)

```

```

Epoch 19/40
11504/11504 ————— 33s 3ms/step - accuracy: 0.9168 - loss: 0.2056 - val_accuracy: 0.9064 - val_loss: 0.2335
Epoch 20/40
11504/11504 ————— 32s 3ms/step - accuracy: 0.9181 - loss: 0.2018 - val_accuracy: 0.9088 - val_loss: 0.2263
Epoch 21/40
11504/11504 ————— 32s 3ms/step - accuracy: 0.9195 - loss: 0.1979 - val_accuracy: 0.9120 - val_loss: 0.2197
Epoch 22/40
11504/11504 ————— 32s 3ms/step - accuracy: 0.9211 - loss: 0.1952 - val_accuracy: 0.9086 - val_loss: 0.2282
Epoch 23/40
11504/11504 ————— 32s 3ms/step - accuracy: 0.9222 - loss: 0.1936 - val_accuracy: 0.9127 - val_loss: 0.2163
Epoch 24/40
11504/11504 ————— 34s 3ms/step - accuracy: 0.9230 - loss: 0.1903 - val_accuracy: 0.9168 - val_loss: 0.2096
Epoch 25/40
11504/11504 ————— 33s 3ms/step - accuracy: 0.9242 - loss: 0.1881 - val_accuracy: 0.9157 - val_loss: 0.2107
Epoch 26/40
11504/11504 ————— 32s 3ms/step - accuracy: 0.9247 - loss: 0.1858 - val_accuracy: 0.9147 - val_loss: 0.2113
Epoch 27/40
11504/11504 ————— 33s 3ms/step - accuracy: 0.9260 - loss: 0.1826 - val_accuracy: 0.9140 - val_loss: 0.2144

```

Figure 30. Training for Adam

The final Adam model is trained using the tuned learning rate, batch size, and epoch count. The returned history object captures the full optimisation trace, and the corresponding Adam loss curve provides a visual diagnostic of convergence and generalisation stability.

6. Comparison and Best Individual Model

6.1 Compare SGD vs Adam validation peaks

```

best_sgd_val = max(history_sgd_final.history["val_accuracy"])
best_adam_val = max(history_adam_final.history["val_accuracy"])

print("Best SGD validation accuracy:", best_sgd_val)
print("Best Adam validation accuracy:", best_adam_val)

```

```
Best SGD validation accuracy: 0.9082819223403931
Best Adam validation accuracy: 0.9168224334716797
```

Figure 31. SDG and Adam Validation Accuracy

The maximum validation accuracy achieved by the tuned SGD and tuned Adam models is compared. By taking the peak validation performance from each final tuning history, the notebook uses a consistent criterion for selecting the superior optimiser under equivalently tuned conditions. The printed results show that Adam attains a higher validation accuracy than SGD, indicating that Adam's adaptive learning-rate mechanism yields better generalisation for this task. On this basis, Adam is identified as the best individual model configuration as it has an accuracy of 0.9168 to carry forward into final evaluation and ensemble construction.

6.2 Compare Training Time for SGD and Adam

```
# Compare Training Time for SGD and Adam

import time

# ===== Measure SGD Training Time =====

start_sgd = time.time()

model_sgd_final = build_model("sgd", best_lr_sgd)
history_sgd = model_sgd_final.fit(
    X_train_full, y_train_full,
    epochs=best_epoch_sgd,
    batch_size=best_bs_sgd,
    validation_split=0.2,
```



```
callbacks=[es],
verbose=0
)

end_sgd = time.time()
sgd_time = end_sgd - start_sgd
print(f"SGD Training Time: {sgd_time:.2f} seconds")

# ===== Measure Adam Training Time =====
start_adam = time.time()

model_adam_final = build_model("adam", best_lr_adam)
history_adam = model_adam_final.fit(
    X_train_full, y_train_full,
    epochs=best_epoch_adam,
    batch_size=best_bs_adam,
    validation_split=0.2,
    callbacks=[es],
    verbose=0
)
```

```
end_adam = time.time()

adam_time = end_adam - start_adam

print(f"Adam Training Time: {adam_time:.2f} seconds")


# ===== Summary =====

print("\n=== Training Time Comparison ===")

print(f"SGD : {sgd_time:.2f} seconds")

print(f"Adam : {adam_time:.2f} seconds")


if sgd_time > adam_time:

    print("→ Adam trains faster than SGD")

else:

    print("→ SGD trains faster than Adam")
```

```
SGD Training Time: 664.77 seconds
Adam Training Time: 942.38 seconds

=== Training Time Comparison ===
SGD : 664.77 seconds
Adam : 942.38 seconds
→ SGD trains faster than Adam
```

Figure 32. SDG and Adam Training Time

This computational efficiency for the final tuned SGD and Adam models is measured. The SDG has 664.77 seconds of training time

while Adam has 942.38 seconds of training time. Thus, SGD has a faster training time than Adam.

6.3 Training and loss curve for SGD

```
# Training and loss curve for SGD
```

```
import matplotlib.pyplot as plt
```

```
# ----- Accuracy Curve -----
```

```
plt.plot(history_sgd.history['accuracy'], label="SGD Train Acc")
```

```
plt.plot(history_sgd.history['val_accuracy'], label="SGD Val Acc")
```

```
plt.legend()
```

```
plt.title("SGD Accuracy Curve")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

```
# ----- Loss Curve -----
```

```
plt.plot(history_sgd.history['loss'], label="SGD Train Loss")
```

```
plt.plot(history_sgd.history['val_loss'], label="SGD Val Loss")
```

```
plt.legend()
```

```
plt.title("SGD Loss Curve")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```

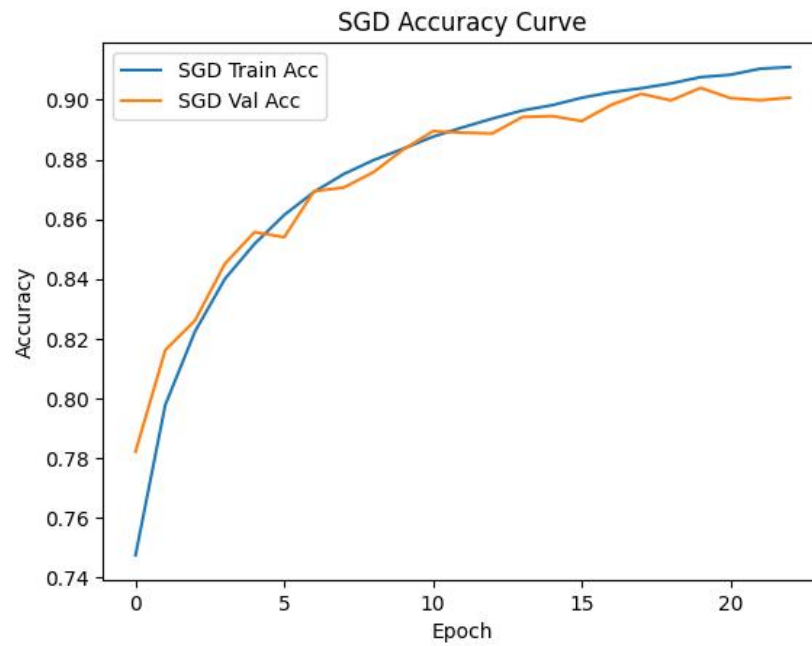


Figure 33. SDG Accuracy Curve

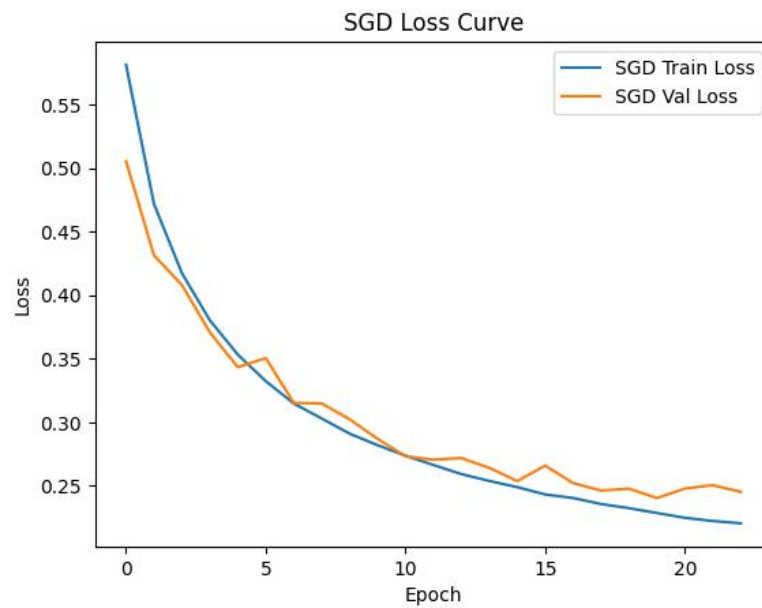


Figure 34. SDG Loss Curve

The learning behaviour of the tuned SGD model is visualised by plotting both accuracy and loss across training epochs for the training and validation splits.

6.4 Training and loss curve for Adam

```
# Training and loss curve for Adam

import matplotlib.pyplot as plt

# ----- Accuracy Curve -----

plt.plot(history_adam.history['accuracy'], label="Adam Train Acc")
plt.plot(history_adam.history['val_accuracy'], label="Adam Val Acc")
plt.legend()

plt.title("Adam Accuracy Curve")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.show()

# ----- Loss Curve -----

plt.plot(history_adam.history['loss'], label="Adam Train Loss")
plt.plot(history_adam.history['val_loss'], label="Adam Val Loss")
plt.legend()

plt.title("Adam Loss Curve")

plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```

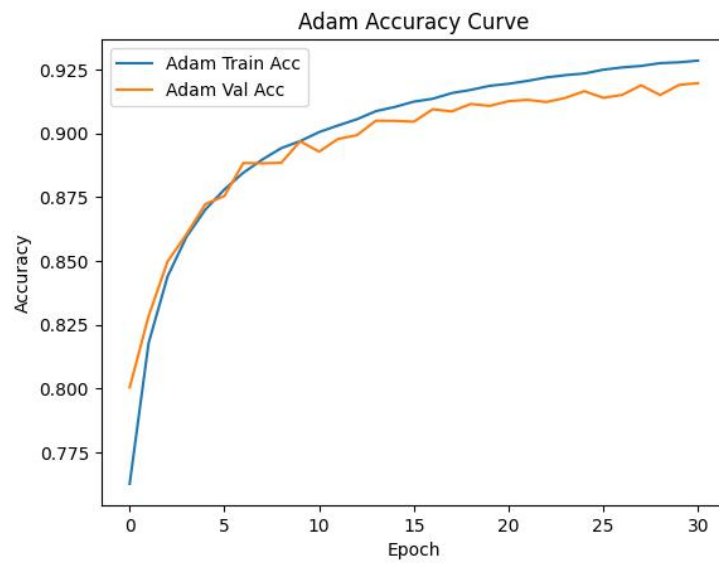


Figure 35. Adam Accuracy Curve

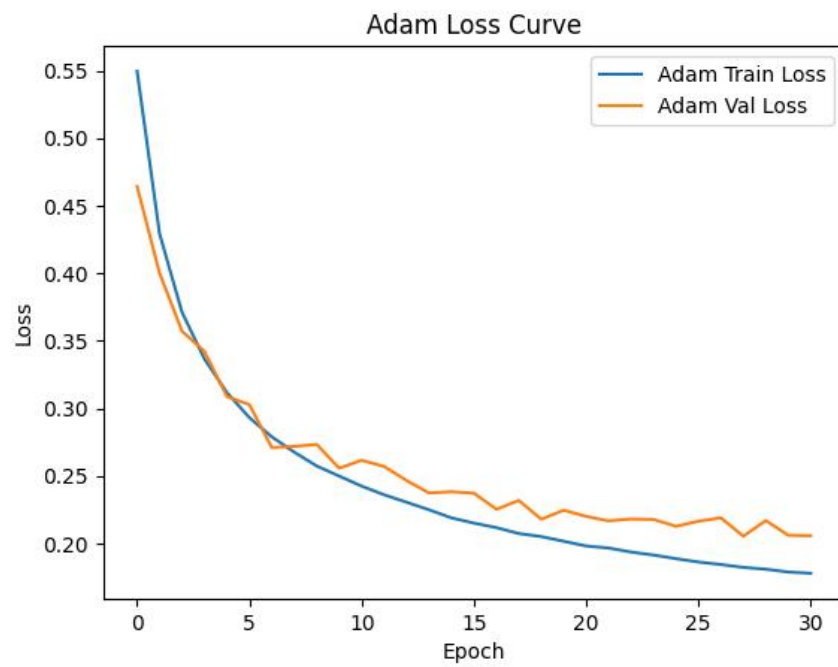


Figure 36. Adam Loss Curve

Adam model's optimisation trajectory is shown by plotting training versus validation accuracy and loss across epochs.

6.5 Save and reload models

```
# Save tuned models
model_sgd_final.save("model_sgd_final.h5")
model_adam_final.save("model_adam_final.h5")

# Reload models
from tensorflow.keras.models import load_model
sgd_reloaded = load_model("model_sgd_final.h5")
adam_reloaded = load_model("model_adam_final.h5")

print("SGD model reloaded successfully.")
print("Adam model reloaded successfully.")
```

The tuned model is saved and reload as .h5 format.

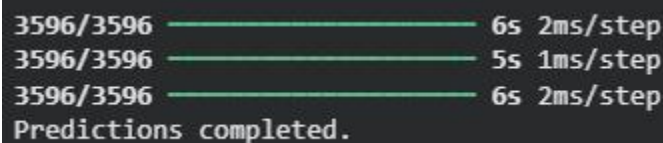
7. Voting Ensemble

7.1 Generate predictions and probabilities

```
import numpy as np

pred_base = np.argmax(baseline_loaded.predict(X_test_full), axis=1)
pred_sgd = np.argmax(sgd_loaded.predict(X_test_full), axis=1)
pred_adam = np.argmax(adam_loaded.predict(X_test_full), axis=1)

print("Predictions completed.")
```



```
3596/3596 ————— 6s 2ms/step
3596/3596 ————— 5s 1ms/step
3596/3596 ————— 6s 2ms/step
Predictions completed.
```

Figure 37. Generate Predictions and Probabilities

Comparable outputs from each trained model are generated on the same held-out test set. For every network, predicted class probabilities are first produced via predict, then converted into discrete class labels using argmax.

7.2 Hard voting ensemble

```
# Combine into matrix shape: (num_samples, 3)
preds = np.vstack([pred_base, pred_sgd, pred_adam]).T

# Hard vote using mode
from scipy.stats import mode
hard_vote_pred, _ = mode(preds, axis=1)

hard_vote_pred = hard_vote_pred.flatten()

print("Hard voting completed.")
```

Hard voting combines models by treating each one as an equal voter. The predicted labels from the baseline, tuned SGD, and tuned Adam models are stacked, and the final class for each sample is chosen as the majority decision. This approach leverages diversity among models, where one model's errors may be offset by correct votes from others.

7.3 Soft voting ensemble

```
# Get actual probability outputs

p_base = baseline_loaded.predict(X_test_full)

p_sgd = sgd_loaded.predict(X_test_full)

p_adam = adam_loaded.predict(X_test_full)


# Average probabilities

soft_prob = (p_base + p_sgd + p_adam) / 3
```



```
soft_vote_pred = np.argmax(soft_prob, axis=1)

print("Soft voting completed.")
```

Soft voting aggregates models at the probability level instead of the label level. By averaging class probability vectors and selecting the class with the highest mean probability, the ensemble accounts for prediction confidence as well as class choice. It yields better performance than hard voting because it can resolve close disagreements by weighting models' certainty, rather than just counting votes.

7.4 Accuracy comparison across models and ensembles

```
from sklearn.metrics import accuracy_score

acc_base = accuracy_score(y_test_full, pred_base)
acc_sgd = accuracy_score(y_test_full, pred_sgd)
acc_adam = accuracy_score(y_test_full, pred_adam)

acc_hard = accuracy_score(y_test_full, hard_vote_pred)
acc_soft = accuracy_score(y_test_full, soft_vote_pred)

print("=== Accuracy Comparison ===")
print(f"Baseline MLP Accuracy : {acc_base:.4f}")
print(f"SGD Best Accuracy      : {acc_sgd:.4f}")
print(f"Adam Best Accuracy       : {acc_adam:.4f}")
```

```
print(f"Ensemble Hard Voting : {acc_hard:.4f}")  
print(f"Ensemble Soft Voting : {acc_soft:.4f}")
```

```
=== Accuracy Comparison ===  
Baseline MLP Accuracy : 0.9061  
SGD Best Accuracy : 0.9032  
Adam Best Accuracy : 0.9195  
Ensemble Hard Voting : 0.9290  
Ensemble Soft Voting : 0.9328
```

Figure 38. Accuracy comparison across models and ensembles

The final test accuracy for each individual model and both ensemble strategies is computed and reported using a consistent metric. The Baseline MLP has an accuracy of 0.9061, SGD has the accuracy of 0.9032, Adam has the accuracy of 0.9195. The ensemble of hard voting has 0.9290 and ensemble hard voting has a soft voting of 0.9328.

7.8 Identify best individual MLP and select its predictions

```
# Only compare individual MLP models  
  
acc_results_mlp = {  
    "baseline": acc_base,  
    "sgd": acc_sgd,  
    "adam": acc_adam  
}  
  
best_model_name = max(acc_results_mlp,  
key=acc_results_mlp.get)  
  
print("Best individual MLP model:", best_model_name)
```

```
# Pick prediction array of the best individual model
```

```
best_pred = {  
    "baseline": pred_base,  
    "sgd": pred_sgd,  
    "adam": pred_adam  
}[best_model_name]
```

Model selection is performed among the three individual MLP variants only, excluding ensemble methods to ensure a fair “single-model best” decision. As a result, the best MLP Model is Adam.

7.9 Confusion matrix heatmap and values

```
from sklearn.metrics import confusion_matrix  
  
import seaborn as sns  
  
import matplotlib.pyplot as plt  
  
# Specify labels to match target values 1-7  
labels = [1, 2, 3, 4, 5, 6, 7]  
  
# Compute confusion matrix with correct labels  
cm = confusion_matrix(y_test_full, best_pred, labels=labels)  
  
plt.figure(figsize=(10, 8))  
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues",
```

```

xticklabels=labels, yticklabels=labels)

plt.title(f"Confusion Matrix – Best Model ({best_model_name})")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

```



Figure 39. Confusion Matrix for Adam (Best Model)

The predictions from the tuned Adam model on the held-out test set is created, converts probability outputs into class labels, and computes the confusion matrix. The heatmap visualisation highlights a strong diagonal structure, indicating that most samples are correctly classified.

The numeric matrix confirms that the largest off-diagonal counts occur between Classes 1 and 2, which is consistent with their ecological similarity and their dominance in the dataset; because these two classes have many more samples, even a small relative error rate produces large absolute confusion counts.

A secondary confusion pattern appears between Class 3 and Class 6, suggesting partial overlap in their feature distributions. Minority classes (especially Classes 4, 5, 6, and 7) show fewer absolute errors but higher proportional misclassification, a typical consequence of imbalanced training data where the model receives less exposure to those decision regions.

8. Discussion

8.1 Impact of Hyperparameter Changes

The hyperparameter exploration demonstrated that learning rate was the dominant factor affecting optimisation behaviour, particularly for SGD. When the learning rate was set too high, SGD exhibited unstable updates and degraded validation performance, reflecting overshooting during gradient descent. Conversely, overly small learning rates led to slow convergence and weaker peak validation accuracy, suggesting that SGD requires careful calibration to balance stability and speed. In contrast, Adam was noticeably more robust across the tested learning-rate range. Its adaptive per-parameter updates dampened the risk of divergence at higher rates and maintained steady convergence even as the learning rate decreased, which explains why Adam ultimately outperformed SGD under comparable tuning.

Batch size also influenced both convergence dynamics and generalisation. Smaller batch sizes produced slightly better validation outcomes, consistent with the idea that noisier gradient estimates can act as a form of regularisation. However, this came with visibly noisier loss trajectories during training. Larger batch sizes improved computational efficiency and yielded smoother learning curves, but tended to flatten validation improvements, indicating a mild reduction in generalisation. The epoch experiments further highlighted the importance of aligning training duration with optimiser characteristics.

Adam benefited from longer training runs, achieving stronger validation peaks before plateauing. At the same time, early stopping remained essential to prevent overfitting once validation loss ceased improving. Together, these patterns confirm that optimiser choice and its associated hyperparameters interact strongly, and that systematic tuning is required to achieve reliable gains beyond baseline settings.

8.2 Strengths and Limitations

The multilayer perceptron models showed clear strengths on this task. With scaled numerical variables and a compact, correlation-informed feature subset, the networks converged efficiently and achieved strong test accuracy. The MLP architecture was able to capture nonlinear interactions among terrain attributes, soil indicators, hillshade measures, and distance-based predictors, which are not well represented by linear models. This ability to learn complex feature relationships likely contributed to the consistent improvements observed once preprocessing and tuning were applied.

Nevertheless, important limitations remained. Performance was sensitive to preprocessing decisions, particularly scaling and outlier handling, confirming that MLPs can be fragile when numerical magnitudes and distributions are not controlled. The persistent class imbalance also constrained performance, as minority classes received fewer effective training signals, leading to higher proportional misclassification. Additionally, certain cover types—especially those with ecological similarity—remained difficult to separate using tabular cartographic attributes alone. The confusion patterns between Classes 1 and 2, and to a lesser extent between Classes 3 and 6, illustrate that even a well-tuned neural model can struggle when classes overlap intrinsically in feature space.

The ensemble extension addressed some of these weaknesses. By combining baseline, tuned SGD, and tuned Adam networks, voting ensembles reduced variance and converted complementary decision boundaries into a stronger aggregate classifier. Soft voting in particular benefited from probability averaging, allowing more confident models to influence the final decision without requiring explicit weighting. This produced the best overall accuracy in the study. However, ensemble gains were bounded by model similarity.

Because all three networks shared the same core architecture and training data, their errors were correlated, limiting the magnitude of achievable improvement. The ensemble also introduced additional computational cost, requiring multiple forward passes and model storage. Despite these trade-offs, the observed performance increase shows that even modest diversity across optimisation settings can yield meaningful generalisation benefits.

9. Conclusion

This study applied multilayer perceptrons to the UCI Forest CoverType multi-class classification problem using a structured pipeline of EDA, preprocessing, baseline modelling, hyperparameter tuning, and ensemble learning. Exploratory analysis revealed heavily skewed numerical distributions, substantial scale differences across continuous predictors, and pronounced class imbalance. These findings motivated targeted preprocessing, including Isolation Forest outlier removal, correlation-based feature selection, and standardisation of numerical variables.

A baseline MLP trained with Adam achieved a test accuracy of 0.9061, establishing a solid reference point. Systematic hyperparameter exploration improved Adam's performance to 0.9195, whereas tuned SGD reached 0.9032, shows that Adam is the stronger individual optimiser for this task. Extending the work with voting ensembles further improved results, with hard voting reaching 0.9290 and soft voting achieving the highest accuracy of 0.9328. Overall, the results show that well-preprocessed and tuned MLPs are effective for forest cover prediction, and that soft-vote ensembling provides a consistent and measurable performance gain over single-network classifiers.